約會一條龍

系統運作說明

電影 + 餐廳 + 大眾運輸，一次排好完整的約會時間軸

2026-10-19 · 對應 GitHub：LittleQQQQ/date（main 與進行中的分支）

1. 這個系統在做什麼

使用者選好約會的商圈、日期、出發站、幾點可以出門、最晚幾點回到家，系統就排出三個完整方案：幾點出門、搭什麼車、看哪一場電影、去哪家餐廳吃飯、先吃還是先看、幾點到家、每人花多少錢。每一段時間都經過程式檢查——接得上場次、趕得上餐廳最後點餐、搭得到末班車、不超過預算。

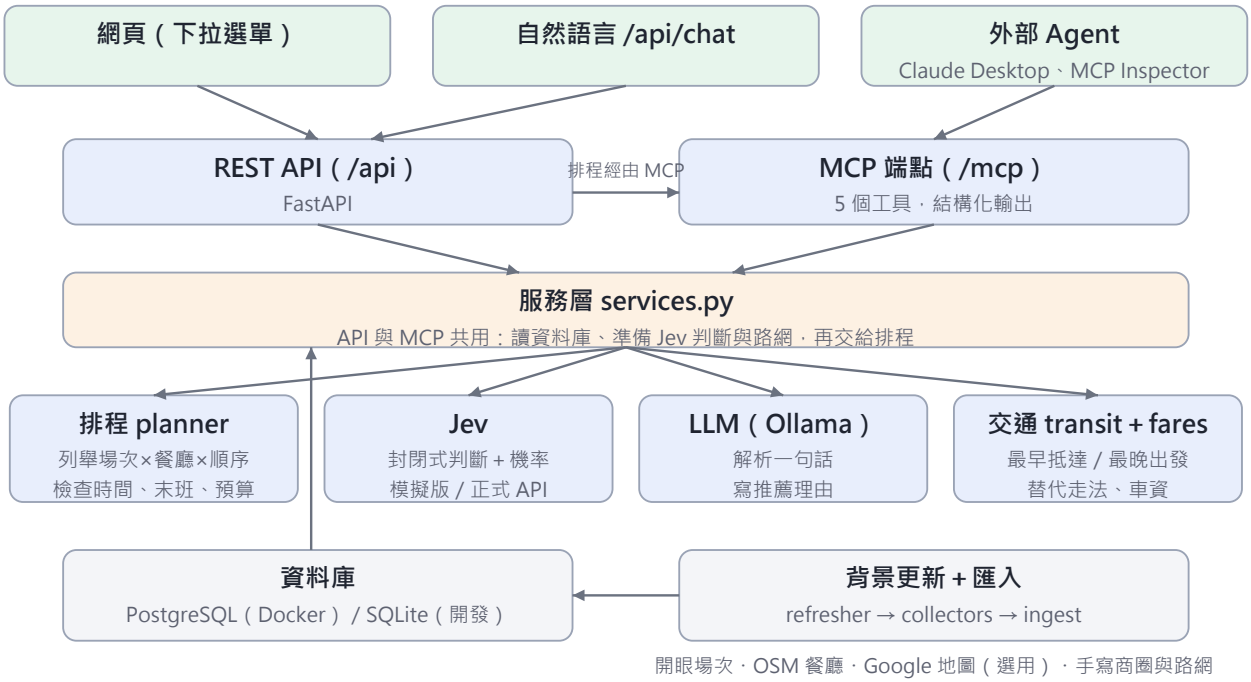
核心設計是**三方分工**，每一方只做自己擅長的事：

角色	負責	不負責
程式	所有計算：時間軸、轉乘路線、末班車、營業時間、預算、車資	主觀判斷（哪部片適合約會）
Jev	封閉式判斷與選擇：從固定選項中給出機率。例如這家店適不適合約會、看哪部片、搭哪種車	產生文字或數字
LLM（本機 Ollama）	語言：把一句話解析成條件、為方案寫一兩句推薦理由	時間、金額（一律由程式的範本寫）

目前開放範圍是**桃園市**（設定 ACTIVE_CITIES）：桃園區、青埔、中壢、八德四個有大型影城的商圈，出發站為桃園車站、中壢車站、高鐵桃園、八德。全台其他縣市的資料仍保留在資料庫與資料檔中，改設定即可恢復。

2. 系統架構

下圖由上而下：使用者從哪裡進來、經過哪些元件、資料存在哪裡、又是從哪裡抓來的。



- **同一個服務**提供網頁、REST API（/api）與 MCP 端點（/mcp）。網頁按「幫我排行程」時，也是透過 MCP 呼叫 plan_date，所以網頁和外部 Agent 拿到的結果一定一致。
- **服務層**是唯一碰資料庫的地方：讀出場次、餐廳、路網，準備好 Jev 的判斷，再交給不碰資料庫的排程模組計算，方便單獨測試。

- **背景更新**在伺服器啟動時補抓資料，之後定時更新；抓到的資料經「匯入」寫進資料庫，排程只讀資料庫，換資料來源不用改排程。

3. 資料從哪裡來

排程只讀資料庫；資料庫裡的資料由下面這些來源提供，再由背景更新定時補抓。

來源	提供什麼	多久更新	備註
開眼電影網（爬蟲）	影城、電影、場次	每天	最多抓今天起 7 天；沒有票價與片型（票價依版本估計、片型由 Jev 判斷）
OpenStreetMap（Overpass）	餐廳	每週	多數店家沒有營業時間與價位，依店的類型估計；沒有評論
Google 地圖 Places API（選用）	餐廳（含評論、營業時間）	排程當下即時查	填了金鑰才用；依條款不存進資料庫；超過每日上限或失敗時改用 OSM
手寫商圈與交通（data/fixture）	商圈、車站、交通路段	伺服器啟動時	車程、班距是估計值；資料檔刪掉的商圈會自動清理（有防呆）
交通部 TDX（分支 feat/tdx-transit）	台鐵、公車真實班次與票價	每週	尚未合併；填 TDX 金鑰才啟用

匯入流程

- 每個資料來源（collector）都輸出同一種格式（Batch），再由 ingest 統一寫進資料庫；同一筆資料用「來源 + 外部代碼」辨認，重抓時更新而不是新增。
- 爬蟲有限速、重試、快取；單頁失敗不中斷整批，整批失敗時保留舊資料。
- 匯入後先請 Jev 回答「不隨使用者條件改變」的題目（約會適合度、料理類型、用餐快慢、影廳、語言、片型），結果依輸入文字的雜湊快取。
- 只有開放範圍（ACTIVE_CITIES）內的商圈會被爬蟲更新；範圍外的資料不刪除、不更新。

4. 排一趟行程的流程

按下「幫我排行程」之後，系統依序做這些事（一般不到 1 秒；有開 LLM 時摘要會多等幾秒）：

步驟	做什麼	誰負責
1 條件	網頁下拉選單，或一句話經 LLM 解析（日期、地點、時間、預算以規則為準）	網頁 / LLM
2 篩選	Jev 判斷影廳、語言、片名、料理、飲食限制、約會適合度；不符合的場次與餐廳先排除	Jev + 程式
3 組合	列舉「場次 × 餐廳 × 先吃或先看」，逐段檢查：開場前到影城、最後點餐、營業時間、末班車、最晚到家、每人預算（含來回車資）	程式
4 交通	去程算「最晚幾點出門」、回程算「最早幾點到家」，考慮轉乘步行、班距、首末班；預算不夠時改找便宜的走法	程式
5 選片	排得進去的片超過 8 部時先挑 8 部，問 Jev 最適合看哪一部	Jev

步驟	做什麼	誰負責
6 選餐廳	對 Jev 前三名的片，各問一次最適合去哪一家（最多 8 家）	Jev
7 前三名	組合信心 = 選片機率 × 選餐廳機率，取前三名；同一組片與餐廳挑緩衝最充裕的場次	程式
8 選車	列出時間接得上、預算付得起的走法（每方向最多 4 種），問 Jev 去程、回程各搭什麼；只有一種就不問	Jev
9 輸出	時間軸、花費明細、「最省」「最穩」標籤；範本寫摘要，LLM 補一兩句不含數字的推薦理由	程式 / LLM

Jev 沒把握時（最高機率低於 0.6）：判斷題相關的場次或餐廳先不排，畫面列出「Jev 不太確定，幫忙確認一下」；選擇題照樣排出方案，另外列出「想換換看嗎」讓使用者改選。使用者的答案會取代 Jev 的猜測並記住。

分支 **feat/jev-llm-assist**（試用中，尚未合併）改成：Jev 沒把握時先把同一題交給本機 LLM 判斷，LLM 也說無法判斷才問使用者；固定題目在匯入時先問好，排程當下最多花約 30 秒等 LLM。設定 JEV_LLM_ASSIST=false 即可恢復上面的做法。

5. 給 Agent 使用：MCP

伺服器啟動後，MCP 端點在 http://主機:8000/mcp（Streamable HTTP）。Agent 依說明先查開放範圍，再排程：

工具	用途
list_areas	目前開放的縣市、有資料的商圈（含可選日期與片單）、可以出發的車站
plan_date	主功能：條件 → 三個方案、Jev 待確認問題（confirmations）、排除原因
search_showtimes	查某天某商圈的場次
find_restaurants	查影城步行範圍內的餐廳與 Jev 判斷
check_route	查某個時間從出發站出發，到商圈各車站怎麼轉乘、最晚幾點到

人在迴圈中：plan_date 的結果裡，confirmations 列出 Jev 沒把握的題目、選項與機率。Agent 問過使用者後，把答案放進 overrides 再呼叫一次，就會得到依使用者選擇重新排好的方案。錯誤會轉成清楚的訊息（例如「目前只開放桃園市」）。

6. 用 Docker 部署

Dockerfile 把網站（含 /api 與 /mcp）打包成映像；docker-compose.yml 一起啟動網站與 PostgreSQL，LLM（Ollama）選用。

容器	內容	資料存放
app	FastAPI 網站 + 背景更新（只開一個 worker）	volume cache：爬蟲的網頁快取
db	PostgreSQL 17	volume pgdata：資料庫
ollama（選用）	本機 LLM，啟動時加 --profile llm	volume ollama：模型

啟動步驟

1. copy .env.docker.example .env，填好 DB_PASSWORD（自己取的密碼）
2. docker compose up -d --build
3. docker compose logs -f app （看進度；/api/status 的 refresh.running 變成 null 就抓完了）
4. 打開 http://localhost:8000

第一次啟動時，網站自動建立資料表、匯入商圈與交通，背景更新再去抓開眼場次與 OSM 餐廳。

本機實際跑一次的結果

- 映像建置成功；啟動後自動建立資料表，匯入 23 個商圈、48 個車站、134 條交通路段。這也是 PostgreSQL 第一次實際跑，匯入、清理、查詢都正常。
- 背景更新約 6.5 分鐘把資料抓齊：OSM 餐廳 229 家（約 3 分鐘）、開眼 7 家影城、46 部片、1,444 個場次（約 3.5 分鐘），沒有失敗。
- 網頁選單只有桃園市的中壢、八德、桃園區、青埔，各有 3～4 天的場次；頁尾顯示 7 家影城、229 家餐廳。
- 排程經由 MCP 正常：桃園車站 → 中壢、中壢車站 → 八德各排出 3 個方案，每次 0.2～0.4 秒；選台北市信義會回「目前只開放桃園市」。
- 容器裡沒開 LLM（OLLAMA_URL 留空），所以摘要用範本；要用 LLM 時加上 ollama 容器。

常用指令

要做什麼	指令
更新程式後重建	git pull，再 docker compose up -d --build
停止（資料保留）/ 連資料刪除	docker compose down / docker compose down -v
備份與搬資料	見 README「換主機」：pg_dump 存在容器裡，再用 docker compose cp 複製出來（不要用 > 轉存，Windows PowerShell 會弄壞備份檔）
資料表結構改版	docker compose exec app python -m app.ingest --reset
使用 LLM	用本機的 Ollama：OLLAMA_URL=http://host.docker.internal:11434（推薦）；或用容器：OLLAMA_URL=http://ollama:11434 並加 --profile llm
對外網址	固定網址：--profile tunnel（Cloudflare 具名通道）；臨時網址：--profile quick-tunnel

兩台電腦輪流當主機：同一時間只開一台。舊主機 docker compose --profile tunnel down；新主機 git pull，再 docker compose --profile tunnel up -d --build。每台各自一份資料庫，啟動後自己抓資料。完整步驟見 README「換主機（兩台電腦輪流當主機）」。

要公開給別人連線前：前面加反向代理（例如 Caddy）處理 HTTPS 與網域，並補上登入、呼叫頻率限制與 MCP 存取權杖（目前都還沒有）。

7. 目前的分支，以及在另一台電腦接續

分支	內容	狀態
main	爬蟲、全台商圈、Jev 選項上限、Jev 選車與估計車資、範圍縮小到桃園、聊天解析修正、Google 地圖餐廳、開眼 7 天、Docker 部署	穩定版
docs/switch-host	換主機步驟、對外網址（Cloudflare 通道）、容器使用本機 Ollama、這份 PDF 的更新	已推上 GitHub，待合併
feat/jev-llm-assist	Jev 沒把握時先請 LLM 判斷（JEV_LLM_ASSIST）	已推上 GitHub，試用中
feat/tdx-transit	交通部 TDX 真實班次與票價	在 GitHub，尚未用真實金鑰驗證

在另一台電腦接續（Windows）

```
git clone https://github.com/LittleQQQQ/date.git
cd date
git checkout 分支名稱 （要接著做某個分支時才需要；不切就是 main）
py -3.13 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pytest -q （測試完全離線）
.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000
```

- 金鑰（TDX、Google Places、正式 Jev API）放在 .env，不會進 git；新電腦要自己建一份。
- 本機的 dev.db 不在 git 裡；新電腦第一次啟動會自動匯入商圈與交通，再由背景更新抓場次與餐廳。
- 要當主機讓別人連線：只要裝 Git 與 Docker Desktop，照 README「換主機（兩台電腦輪流當主機）」操作。

8. 目前的限制與下一步

- Jev 仍是模擬版（關鍵字規則），正式 API 的格式是假設的；要和 Jev 確認一題最多幾個選項。
- 交通是手寫估計值（TDX 分支待驗證）；票價、OSM 餐廳的營業時間與價位多為估計。
- API 與 MCP 沒有登入、呼叫頻率限制；部署到網路上前要補。
- 網頁還沒有聊天輸入框（自然語言只有 /api/chat）；外部 Agent 的多輪流程還沒實際示範過。